

```
def getArea(self):
    theArea = self.width * self.height
    return theArea
```

Listing 10-11: The `Rectangle` class that implements `clickedInside()` and `getArea()`, but not `draw()`

As a demonstration, this class mistakenly does not contain a `draw()` method. [Listing 10-12](#) shows a modified version of the main code that includes the creation of `Rectangle` objects.

File: `InheritedShapes/Main_ShapesWithRectangle.py`

```
shapesList = []
shapeClassesTuple = ('Square', 'Circle', 'Triangle', 'Rectangle')
for i in range(0, N_SHAPES):
    randomlyChosenClass = random.choice(shapeClassesTuple)
    oShape = randomlyChosenClass(window, WINDOW_WIDTH, WINDOW_HEIGHT)
    shapesList.append(oShape)
```

Listing 10-12: The main code that randomly creates `SquareS`, `CircleS`, `TriangleS`, and `RectangleS`

When this code attempts to create a `Rectangle` object, Python generates this error message:

```
TypeError: Can't instantiate abstract class Rectangle with abstract method draw
```

This tells us that we cannot instantiate a `Rectangle` object because we did not write a `draw()` method in our `Rectangle` class. Adding a `draw()` method to the `Rectangle` class (with appropriate code to draw the rectangle) fixes the error.

How `pygwidgets` Uses Inheritance

The `pygwidgets` module uses inheritance to share common code. For example, consider the two button classes we discussed in Chapter 7: `TextButton` and `CustomButton`. The `TextButton` class requires a string to be used as a label on the button, while the `CustomButton` class requires you to supply your own art. The